

EXPERIMENT 3

Best Practices for Code Organization — Scripts, Modularization & Documentation

Aim

To organise a Python project into reusable modules and scripts, and to document code using docstrings and inline comments following industry standards (PEP 8, PEP 257).

Theory

Well-organised code is easier to read, test, reuse and maintain. Modularization is the practice of splitting a program into logically cohesive files (modules) and packages, each responsible for a single concern. In Python, any .py file is a module and a directory containing an __init__.py is a package.

The separation of a reusable library (functions/classes with no side effects at import) from an executable script (the entry point) is achieved with the 'if __name__ == "__main__":' guard. This ensures code runs only when the file is executed directly, not when imported.

Documentation has two layers: docstrings (PEP 257) — triple-quoted strings at the top of modules, classes and functions that describe purpose, parameters and return values — and inline comments explaining non-obvious logic. Consistent style is enforced by PEP 8 and tools such as flake8/black.

Software / Tools Required

Python 3.10+, VS Code, flake8 or black (optional linter/formatter).

Procedure (Step-by-Step)

Step 1. Create a clean project structure separating source, scripts and tests.

```
mkdir -p mlproject/src mlproject/scripts mlproject/tests
cd mlproject
```

Step 2. Create a reusable utility module with well-documented functions.

```
# src/mathutils.py
"""Reusable numeric helper functions."""

def mean(values):
    """Return the arithmetic mean of a non-empty list of numbers.

    Args:
        values (list[float]): numbers to average.
    Returns:
        float: the mean.
    """
    return sum(values) / len(values)

def normalize(values):
    """Scale values to the range [0, 1] using min-max normalization."""
    lo, hi = min(values), max(values)
    return [(v - lo) / (hi - lo) for v in values]
```

Step 3. Create an executable script that imports and uses the module behind a main-guard.

```
# scripts/run.py
"""Entry point that demonstrates the mathutils module."""
import sys, os
```

```

sys.path.append(os.path.join(os.path.dirname(__file__), "..", "src"))
from mathutils import mean, normalize

def main():
    data = [10, 20, 30, 40]
    print("Mean:", mean(data))
    print("Normalized:", normalize(data))

if __name__ == "__main__":
    main()

```

Step 4. Run the script and observe the output.

```
python scripts/run.py
```

Step 5. Turn 'src' into a proper package by adding an `__init__.py`.

```
touch src/__init__.py
```

Step 6. Install and run a linter to check PEP 8 compliance.

```

pip install flake8
flake8 src/ scripts/ --max-line-length=100

```

Step 7. Auto-format the code with black to enforce a consistent style.

```

pip install black
black src/ scripts/

```

Step 8. Add a module-level README documenting how to run the project.

```
echo "# ML Project\nRun with: python scripts/run.py" > README.md
```

Step 9. Inspect the auto-generated help built from your docstrings.

```
python -c "import sys; sys.path.append('src'); import mathutils; help(mathutils)"
```

Step 10. Verify the final directory layout.

```
find . -type f -not -path "*/.*"
```

Expected Output

The script prints the computed mean and normalized list. flake8 reports zero style violations, and 'help(mathutils)' displays the docstrings, confirming the code is modular, runnable and self-documented.

Conclusion

A Python project was organised into modules, packages and scripts with proper docstrings and PEP 8 formatting, demonstrating industry-standard code organisation and documentation practices.

Viva Questions

1. What does the 'if `__name__ == "__main__"`' guard do?
2. Differentiate between a module and a package in Python.
3. What is the purpose of a docstring and how does it differ from a comment?
4. What is PEP 8 and why does it matter?
5. How does modularization improve testability?